

AWS DATA ENGINEER ASSOCIATE

Day 1: Ingestion Patterns (Batch & Streaming)

S3 Ingestion | Kinesis | Glue Crawlers | Orchestration Patterns

Duration	2 Hours (Part of 16-Hour Course)
AWS Exam Domain	Data Ingestion and Transformation
Key Services	S3, Kinesis, Glue, Step Functions, EventBridge

1. Overview: AWS Data Ingestion Landscape

Data ingestion is the foundation of any data engineering pipeline. On AWS, data ingestion refers to the process of collecting, transporting, and loading data from various sources into storage or processing systems. This day covers two primary ingestion paradigms: batch ingestion (scheduled, bulk data movement) and streaming ingestion (continuous, real-time data flow).

1.1 Why Ingestion Architecture Matters

Choosing the right ingestion pattern impacts downstream data quality, cost, latency, and scalability. A poorly designed ingestion layer leads to:

- Data loss or duplication during transit
- Bottlenecks that delay analytics and ML pipelines
- Excessive costs from over-engineered or under-optimized transfers
- Compliance issues if sensitive data isn't handled at ingestion time

1.2 Batch vs Streaming: High-Level Comparison

Dimension	Batch Ingestion	Streaming Ingestion
Latency	Minutes to hours	Milliseconds to seconds
Data Volume	Large bulk transfers	Continuous small records
AWS Services	S3, Glue, DMS, Snow*	Kinesis, MSK, DMS CDC
Use Cases	ETL, reporting, daily loads	Fraud detection, IoT, clickstream
Cost Model	Lower per-GB (batch pricing)	Higher (always-on shards/brokers)

Dimension	Batch Ingestion	Streaming Ingestion
Complexity	Lower operational overhead	Higher (ordering, deduplication)
Reprocessing	Easy (re-run job)	More complex (replay from offset)

2. Amazon S3 Ingestion

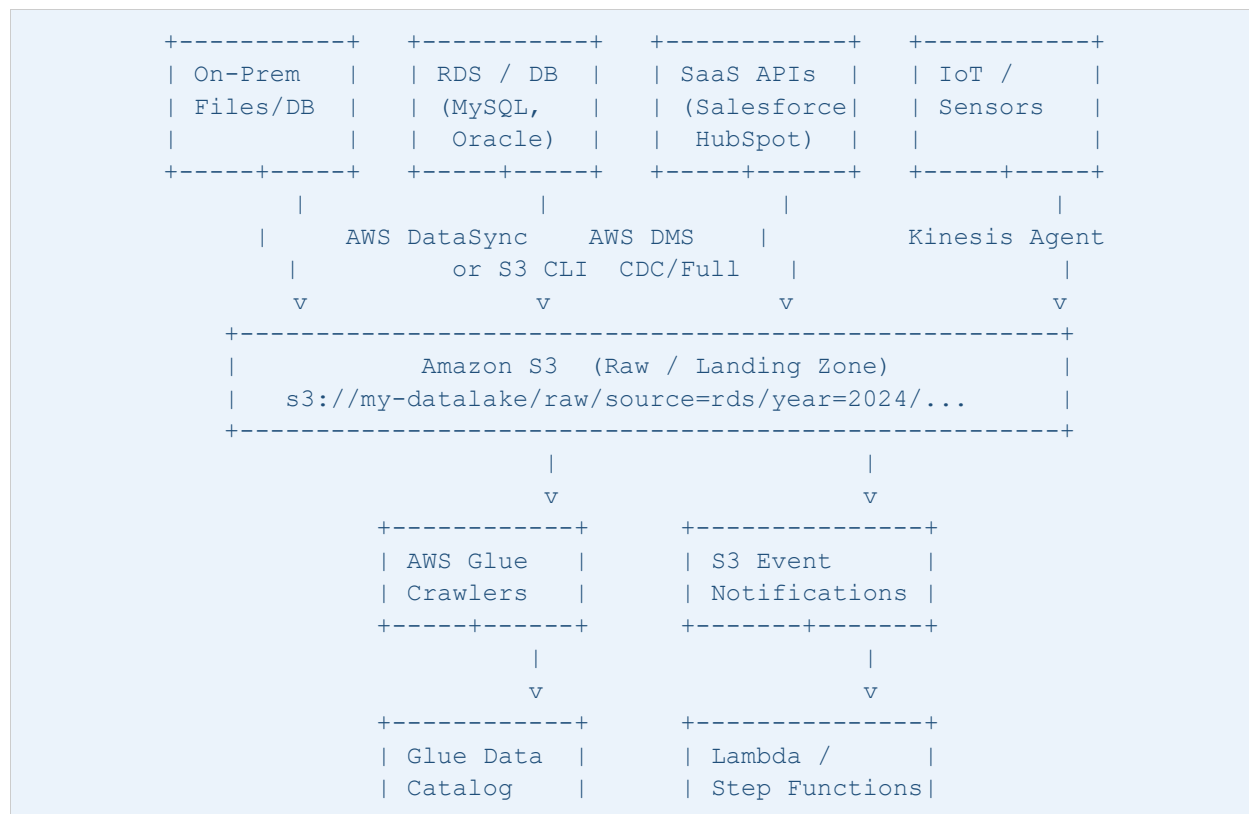
Amazon S3 (Simple Storage Service) is the primary landing zone for batch data on AWS. It serves as a durable, scalable, and cost-effective storage layer that integrates with virtually every AWS analytics service.

2.1 S3 Core Concepts for Data Engineers

- Buckets: Top-level namespace containers. Each bucket has a globally unique name.
- Objects: Files stored in buckets. Max size 5TB; multipart upload recommended for >100MB.
- Prefixes: Simulate folder structure (e.g., s3://bucket/year=2024/month=01/). Critical for partitioning.
- Storage Classes: S3 Standard, Intelligent-Tiering, Standard-IA, Glacier, Glacier Deep Archive.
- Versioning: Enables recovery from accidental deletes/overwrites. Required for compliance use cases.

2.2 S3 Ingestion Architecture Diagram

The following diagram illustrates a typical multi-source batch ingestion architecture using S3 as the data lake landing zone:



+-----+ +-----+

2.3 S3 Ingestion Methods

2.3.1 AWS CLI / SDK Direct Upload

The simplest form of ingestion — push files directly to S3 using the AWS CLI or Boto3 SDK:

```
# AWS CLI: Upload single file
aws s3 cp local_data.csv s3://my-datalake/raw/source=crm/year=2024/month=01/

# AWS CLI: Sync directory (only new/changed files)
aws s3 sync ./data/ s3://my-datalake/raw/source=local/ --exclude '*.tmp'

# Python Boto3: Upload with metadata tags
import boto3
s3 = boto3.client('s3')
s3.upload_file(
    Filename='data.parquet',
    Bucket='my-datalake',
    Key='raw/source=api/year=2024/data.parquet',
    ExtraArgs={'Metadata': {'source': 'api', 'ingestion_date': '2024-01-15'}}
)
```

2.3.2 AWS DataSync

DataSync automates and accelerates data transfers between on-premises storage (NFS, SMB, HDFS) and S3. It is ideal for large-scale migrations and ongoing sync tasks.

- Transfers up to 10x faster than open-source tools via parallel multi-threaded transfers
- Supports scheduling, filtering, bandwidth throttling, and data integrity verification
- Integrated with CloudWatch for monitoring and SNS for alerts

2.3.3 S3 Multipart Upload (Large Files)

For files larger than 100MB, use multipart upload. Minimum part size is 5MB. Benefits include parallelism, resume capability, and fault tolerance.

```
# Boto3 Multipart Upload Example
import boto3
from boto3.s3.transfer import TransferConfig

config = TransferConfig(
    multipart_threshold=1024 * 25,  # 25MB threshold
    max_concurrency=10,              # Parallel threads
    multipart_chunksize=1024 * 25,   # 25MB chunks
    use_threads=True
)
```

```
s3 = boto3.client('s3')
s3.upload_file('large_file.parquet', 'my-datalake', 'raw/large_file.parquet',
              Config=config)
```

2.4 S3 Partitioning Strategy

Best Practice: Hive-Style Partitioning

Use Hive-style partition keys: `s3://bucket/table/year=2024/month=01/day=15/`

This format is auto-discovered by Glue Crawlers and Athena partition projection.

Partition by high-cardinality fields queried most often (date, region, source).

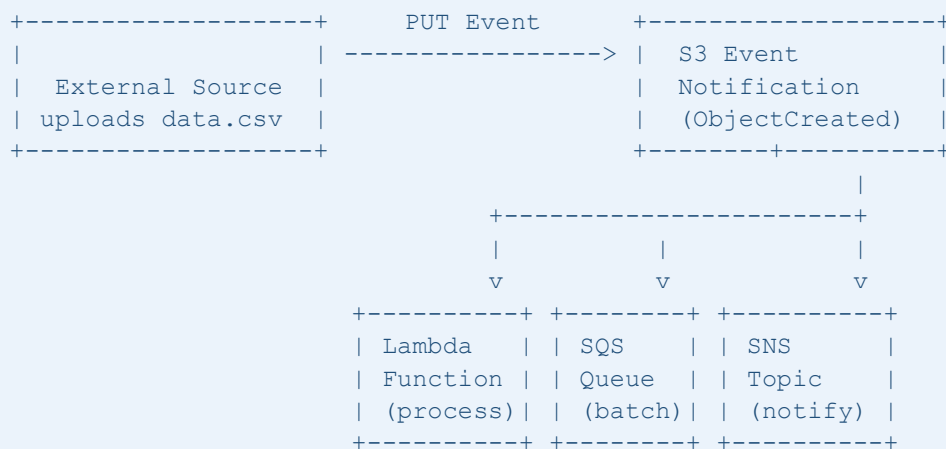
Avoid over-partitioning: too many small partitions increase metadata overhead.

Target partition size: 128MB - 1GB for optimal Spark/Athena performance.

Partition Strategy	Example Path	Best For
Date partitioned	<code>s3://bucket/events/year=2024/month=01/day=15/</code>	Time-series analytics
Source + Date	<code>s3://bucket/raw/source=salesforce/year=2024/</code>	Multi-source data lakes
Region partitioned	<code>s3://bucket/orders/region=us-east-1/</code>	Geo-distributed queries
Event type	<code>s3://bucket/logs/event_type=error/year=2024/</code>	Filtered event processing

2.5 S3 Event Notifications

S3 can trigger downstream processing automatically when objects are created, modified, or deleted. This is a key pattern for event-driven ingestion pipelines.



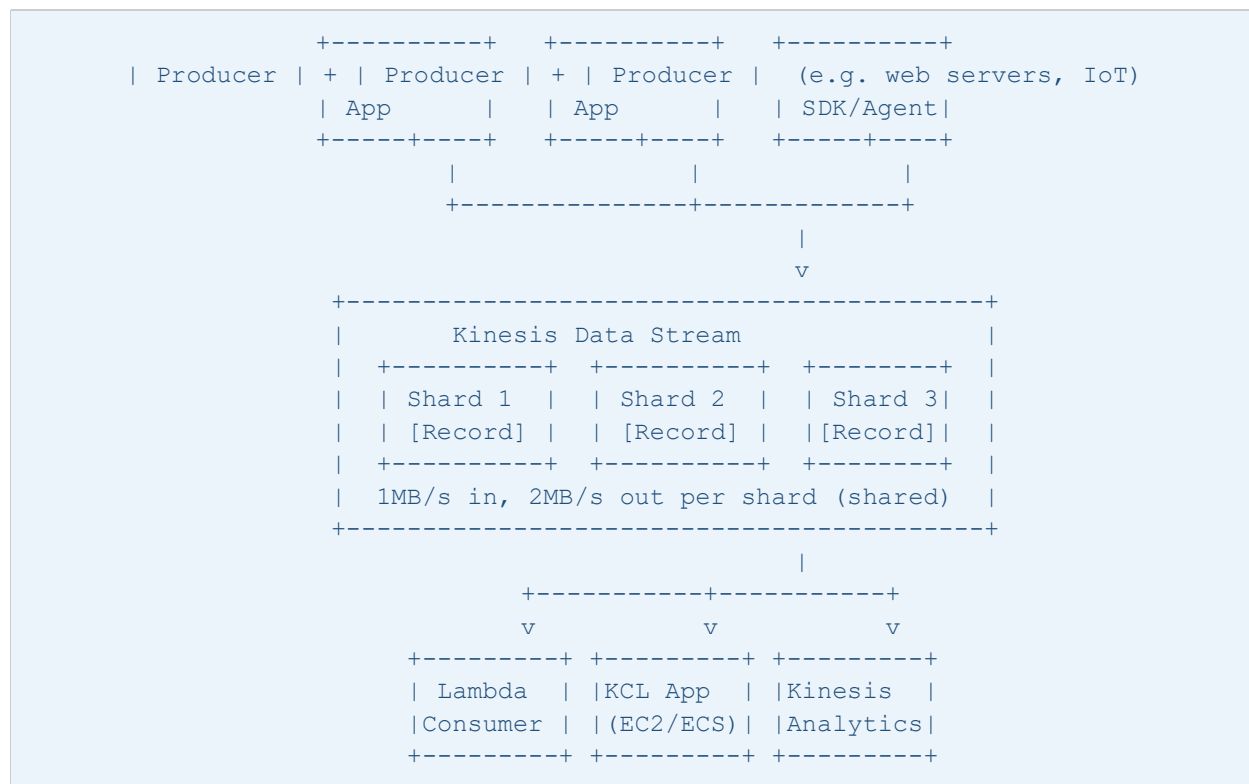
3. Amazon Kinesis: Streaming Ingestion

Amazon Kinesis is a family of managed services for real-time data streaming. It enables you to ingest, process, and analyze streaming data at any scale.

3.1 Kinesis Family Overview

Service	Purpose	When to Use
Kinesis Data Streams (KDS)	Real-time data streaming platform	Custom processing, low latency, replay
Kinesis Data Firehose	Load streams to data stores	S3, Redshift, OpenSearch, no-code ETL
Kinesis Data Analytics	SQL/Flink on streams	Windowed aggregations, anomaly detection
Kinesis Video Streams	Streaming video ingestion	ML inference on video, media processing

3.2 Kinesis Data Streams (KDS) Deep Dive



3.2.1 Key KDS Concepts

- **Shard:** Base unit of capacity. Each shard = 1 MB/s input + 2 MB/s output (shared) or 2MB/s enhanced fan-out per consumer
- **Partition Key:** String used to route records to specific shards via MD5 hash. Hot shards occur with low-cardinality keys.
- **Sequence Number:** Unique identifier assigned to each record per shard. Used for ordering and checkpointing.
- **Retention Period:** Default 24 hours, extendable to 7 days (standard) or 365 days (long-term). Cost increases with longer retention.
- **Enhanced Fan-Out:** Dedicated 2 MB/s per consumer per shard via HTTP/2 push. Ideal for multiple parallel consumers.

3.2.2 Shard Capacity Planning

Metric	Per Shard Limit	Notes
Write throughput	1 MB/s or 1,000 records/s	Whichever is hit first
Read throughput (shared)	2 MB/s total across all consumers	Default GetRecords
Read throughput (EFO)	2 MB/s per registered consumer	Enhanced Fan-Out
Max record size	1 MB	Compress large records
Shard split/merge	Supported (takes ~30s)	Scale manually or auto

3.2.3 KDS Producer Code

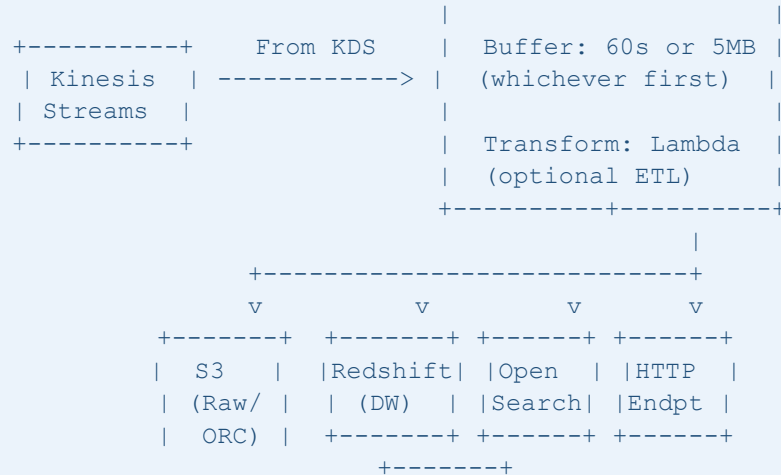
```
import boto3, json

kinesis = boto3.client('kinesis', region_name='us-east-1')

# Batch send (more efficient - up to 500 records per call)
def send_batch(stream_name, records):
    entries = [
        {'Data': json.dumps(r).encode(), 'PartitionKey': r['user_id']}
        for r in records
    ]
    response = kinesis.put_records(StreamName=stream_name, Records=entries)
    failed = response['FailedRecordCount']
    if failed > 0:
        print(f'WARNING: {failed} records failed - implement retry logic')
```

3.3 Kinesis Data Firehose

```
+-----+      Direct PUT      +-----+
| Producer | -----> | Kinesis Firehose |
+-----+              | Delivery Stream |
```

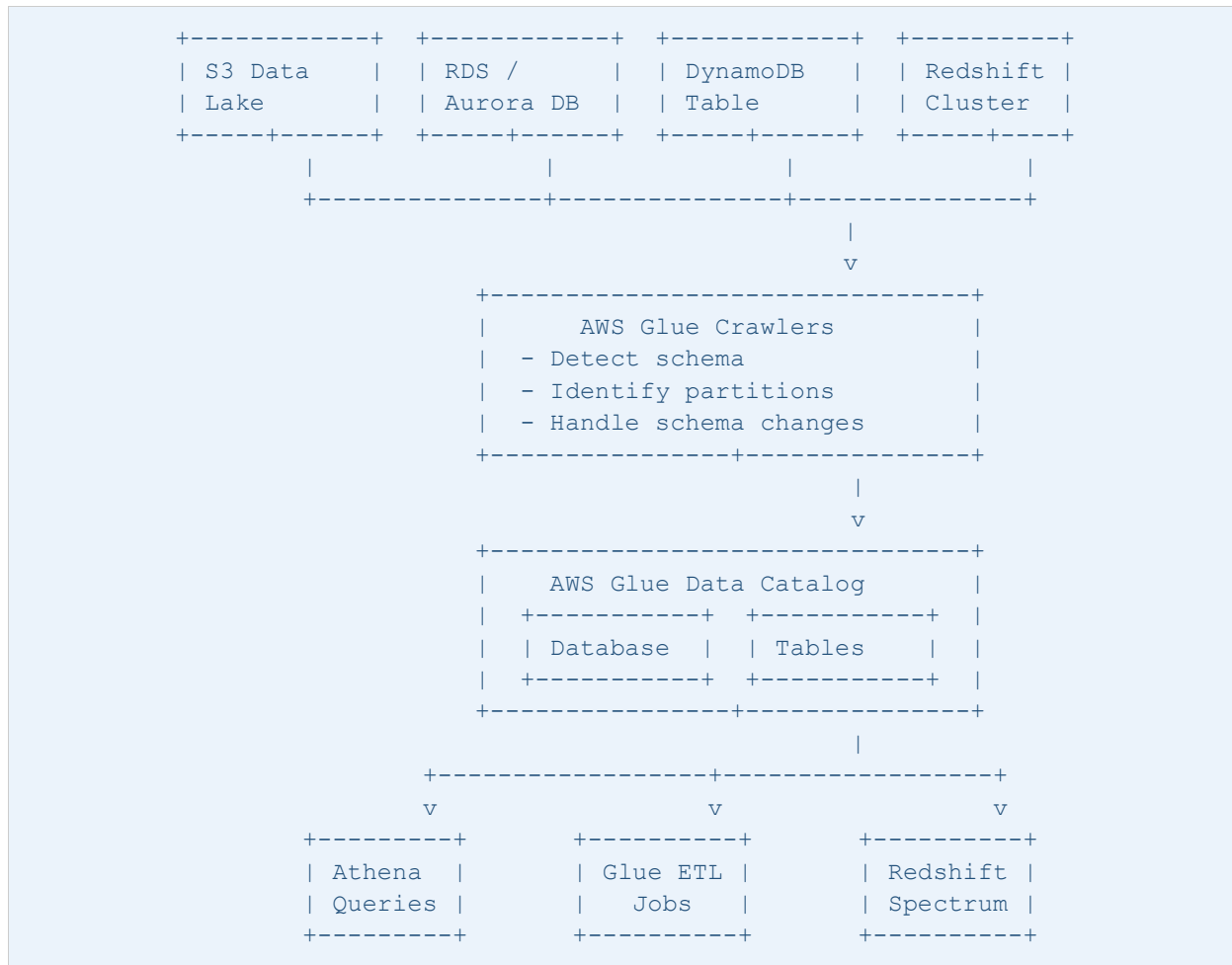


- Destinations: S3, Amazon Redshift, Amazon OpenSearch, Splunk, Datadog, HTTP endpoints
- Buffer hints: Time (60-900s) or size (1-128MB). Lower buffers = lower latency, higher cost.
- Data transformation: Invoke Lambda function inline to filter, mask, or enrich records
- Format conversion: Automatically convert JSON to Parquet or ORC before writing to S3
- Error handling: Failed records go to a configurable S3 error prefix for debugging

4. AWS Glue Crawlers & Data Catalog

AWS Glue Crawlers automatically scan data stores (S3, JDBC databases, DynamoDB), infer schema, and register table metadata in the AWS Glue Data Catalog. The catalog acts as a central metadata repository used by Athena, EMR, Redshift Spectrum, and Glue ETL jobs.

4.1 Glue Data Catalog Architecture



4.2 How Glue Crawlers Work

1. Crawler connects to the data store using a configured IAM role and connection
2. Classifiers are applied to determine file format (CSV, JSON, Parquet, Avro, ORC)
3. Schema is inferred by sampling records from each file/table
4. Partitions are detected from S3 path structure (Hive-style keys like year=2024/)
5. Metadata is written to Glue Data Catalog as a database and table definition
6. On subsequent runs, the crawler detects new partitions and schema changes

4.3 Crawler Configuration Options

Setting	Options	Recommendation
Data Store	S3, JDBC, DynamoDB, Delta Lake	Match to your source
IAM Role	Must have S3 read + Glue write	Least-privilege role
Classifiers	Built-in (CSV/JSON/Parquet) or custom Grok	Use built-in unless custom format
Schedule	On-demand, hourly, daily, cron	Daily for batch; on-demand for ad-hoc
Schema change handling	Add new columns, update schema, or create new table	Log for critical tables
Partition detection	Auto-detect Hive style	Ensure consistent path format

4.4 Partition Management via API

```
# Add new partitions without running full crawler (Athena)
MSCK REPAIR TABLE clickstream_db.events;

# OR use Glue batch create partition API (Python)
import boto3
glue = boto3.client('glue')
glue.batch_create_partition(
    DatabaseName='clickstream_db',
    TableName='events',
    PartitionInputList=[{
        'Values': ['2024', '01', '15'],
        'StorageDescriptor': {
            'Location': 's3://my-datalake/processed/year=2024/month=01/day=15/',
            'InputFormat': 'org.apache.hadoop.mapred.TextInputFormat',
            'SerdeInfo': {'SerializationLibrary':
                'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'}
        }
    }]
)
```

5. Orchestration Patterns

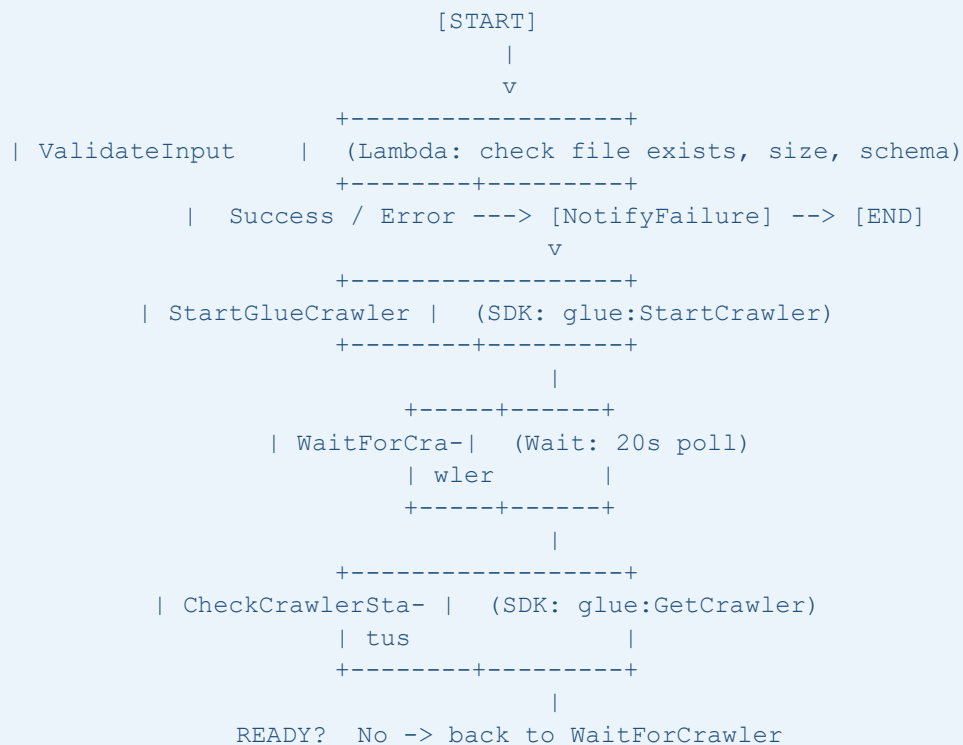
Orchestration coordinates the execution of multiple ingestion and transformation steps in the correct order, handling dependencies, retries, branching logic, and error handling.

5.1 Orchestration Options Comparison

Service	Best For	Key Feature
AWS Step Functions	Complex multi-step workflows with branching	Visual state machine, SDK integrations
Amazon EventBridge	Event-driven, loosely-coupled triggers	Rule-based routing, schedule, patterns
AWS Glue Workflows	Glue-centric ETL DAG orchestration	Native Glue job/crawler chaining
Amazon MWAA (Airflow)	Complex DAG orchestration, Python-native	Managed Apache Airflow
AWS Batch + EventBridge	Containerized batch job scheduling	Batch compute orchestration

5.2 Step Functions State Machine Diagram

Step Functions State Machine: Ingestion Pipeline



```

      | Yes
      v
+-----+
| NotifySuccess | (SNS: publish success)
+-----+
      |
      [END]

```

5.3 EventBridge Scheduling Diagram

Event Sources	EventBridge	Targets
+-----+	+-----+	+-----+
S3 Put	---Event---> Default	--Rule--> Lambda
DynamoDB	---Event---> Event	--Rule--> Step Fn
Kinesis	---Event---> Bus	--Rule--> SNS
+-----+	+-----+	+-----+
+-----+	+-----+	+-----+
Scheduled	---Cron---> Scheduler	
(cron expr)		Rules
+-----+		+-----+

5.4 Glue Workflows

- On-demand or triggered by EventBridge schedules
- Supports conditional triggers (run next job only if previous succeeded)
- Built-in monitoring via Glue console and CloudWatch metrics
- Best for Glue-centric pipelines; use Step Functions for cross-service orchestration

6. Hands-On Labs: Step-by-Step Implementation Guide

This section provides complete, production-quality implementation guides for all four labs. Each lab builds on the previous one. Run all commands in a terminal with AWS CLI v2 configured (aws configure). Commands use bash environment variables set in earlier steps.

Lab Environment Setup

AWS CLI v2 installed: `aws --version` should return 2.x.x

Credentials configured: `aws sts get-caller-identity` should return your Account ID

Python 3.8+: `python3 --version` should return 3.8 or higher

boto3 installed: `pip install boto3`

Region: all labs use us-east-1 for consistency

Important: Run labs in sequence — each lab depends on resources created in previous labs

Lab 1: S3 Data Lake Setup & Event-Driven Ingestion

Objective: Create a versioned S3 data lake bucket with Hive-style partitioned prefixes, attach an S3 event notification to a Lambda function, test with sample data uploads, and apply a lifecycle policy for cost management.

Pre-requisites:

- AWS CLI v2 installed and configured (aws configure) with a user/role having S3FullAccess and LambdaFullAccess
- Python 3.8+ installed locally for producing sample data files
- An AWS region selected – use us-east-1 throughout this lab for consistency

Step 1: Create and configure the S3 data lake bucket

Create an S3 bucket with a globally unique name. Enable versioning so all object versions are retained (required for data recovery and compliance). Also block all public access for security.

```
# Replace 'YOUR_ALIAS' with your name or unique ID (bucket names must be globally unique)
export BUCKET_NAME="my-datalake-lab-YOUR_ALIAS"
export REGION="us-east-1"

# Create the bucket
aws s3api create-bucket \
  --bucket $BUCKET_NAME \
  --region $REGION

# Enable versioning
aws s3api put-bucket-versioning \
  --bucket $BUCKET_NAME \
  --versioning-configuration Status=Enabled
```

```
# Block all public access (security best practice)
aws s3api put-public-access-block \
  --bucket $BUCKET_NAME \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

☒ **Expected Output:**

CreateBucket returns HTTP 200. No output means success for the versioning/block commands.

Step 2: Create the Hive-style folder structure (landing zone prefixes)

S3 does not have real folders – prefixes simulate them. Upload placeholder .keep files to establish the partition structure so downstream crawlers recognise the layout immediately.

```
# Create raw landing zone prefixes for two sources
aws s3api put-object --bucket $BUCKET_NAME --key
raw/source=csv/year=2024/month=01/.keep
aws s3api put-object --bucket $BUCKET_NAME --key
raw/source=api/year=2024/month=01/.keep
aws s3api put-object --bucket $BUCKET_NAME --key
processed/year=2024/month=01/.keep
aws s3api put-object --bucket $BUCKET_NAME --key errors/.keep

# Verify structure
aws s3 ls s3://$BUCKET_NAME/ --recursive
```

☒ **Expected Output:**

You see four .keep objects listed with the full Hive-style prefix paths.

Step 3: Create the Lambda execution IAM role

Lambda needs an IAM role to execute. The role must trust the Lambda service and have permissions to write CloudWatch Logs.

```
# Create the trust policy document
cat > /tmp/lambda-trust.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "lambda.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF

# Create the role
aws iam create-role \
  --role-name S3EventLambdaRole \
  --assume-role-policy-document file:///tmp/lambda-trust.json
```

```
# Attach basic Lambda execution (CloudWatch Logs) policy
aws iam attach-role-policy \
  --role-name S3EventLambdaRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

# Allow Lambda to read from S3 (for validation)
aws iam attach-role-policy \
  --role-name S3EventLambdaRole \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Capture the role ARN for later
LAMBDA_ROLE_ARN=$(aws iam get-role --role-name S3EventLambdaRole --query
'Role.Arn' --output text)
echo "Role ARN: $LAMBDA_ROLE_ARN"
```

☒ Expected Output:

Role ARN printed in the form: arn:aws:iam::123456789012:role/S3EventLambdaRole

Step 4: Write and deploy the Lambda function

The function logs the bucket name, object key, event time and object size whenever a new file is PUT into the bucket. This pattern is the foundation for any event-driven ingestion trigger.

```
# Write the Lambda handler
cat > /tmp/s3_event_handler.py << 'EOF'
import json, logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    for record in event.get('Records', []):
        bucket = record['s3']['bucket']['name']
        key    = record['s3']['object']['key']
        size   = record['s3']['object'].get('size', 0)
        etime  = record['eventTime']
        logger.info(f"NEW OBJECT | bucket={bucket} | key={key} | size={size}B |
time={etime}")
        # TODO: trigger Glue crawler or Step Functions here
    return {'statusCode': 200, 'body': f"Processed {len(event['Records'])}
record(s)"}
EOF

# Package and deploy
cd /tmp && zip s3_handler.zip s3_event_handler.py

aws lambda create-function \
  --function-name s3-ingestion-trigger \
  --runtime python3.12 \
  --role $LAMBDA_ROLE_ARN \
  --handler s3_event_handler.lambda_handler \
  --zip-file fileb:///tmp/s3_handler.zip \
  --timeout 30
```

```
# Capture the Lambda ARN
LAMBDA_ARN=$(aws lambda get-function --function-name s3-ingestion-trigger \
  --query 'Configuration.FunctionArn' --output text)
echo "Lambda ARN: $LAMBDA_ARN"
```

☒ **Expected Output:**

Lambda function created. FunctionArn printed, State should be Active.

Step 5: Grant S3 permission to invoke Lambda

S3 event notifications use a resource-based policy on Lambda to allow the S3 service to invoke the function. Without this step the notification will be silently dropped.

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

aws lambda add-permission \
  --function-name s3-ingestion-trigger \
  --statement-id s3-invoke-permission \
  --action lambda:InvokeFunction \
  --principal s3.amazonaws.com \
  --source-arn arn:aws:s3:::$BUCKET_NAME \
  --source-account $ACCOUNT_ID
```

☒ **Expected Output:**

JSON statement returned confirming the permission was added to the function policy.

Step 6: Configure S3 event notification

Tell S3 to call the Lambda function whenever any object is created (PUT, POST, COPY, or multipart upload complete) under the raw/ prefix.

```
cat > /tmp/notification.json << EOF
{
  "LambdaFunctionConfigurations": [{
    "LambdaFunctionArn": "$LAMBDA_ARN",
    "Events": ["s3:ObjectCreated:*"],
    "Filter": {
      "Key": {
        "FilterRules": [{"Name": "prefix", "Value": "raw/"}]
      }
    }
  }]
}
EOF

aws s3api put-bucket-notification-configuration \
  --bucket $BUCKET_NAME \
  --notification-configuration file:///tmp/notification.json

# Verify the configuration was applied
aws s3api get-bucket-notification-configuration --bucket $BUCKET_NAME
```


☒ Expected Output:

JSON output showing the `LambdaFunctionConfigurations` entry with the correct ARN and prefix filter.

Step 7: Test the end-to-end pipeline

Create a sample CSV file and upload it to the `raw/` prefix. The S3 event fires, Lambda is invoked, and a log entry appears in CloudWatch.

```
# Create a sample CSV
cat > /tmp/sample_events.csv << 'EOF'
user_id,event,page,timestamp
u001,click,home,2024-01-15T10:00:00Z
u002,view,product,2024-01-15T10:01:00Z
u003,purchase,checkout,2024-01-15T10:02:00Z
EOF

# Upload to the raw/ prefix (this fires the S3 event)
aws s3 cp /tmp/sample_events.csv \
  s3://$BUCKET_NAME/raw/source=csv/year=2024/month=01/sample_events.csv

# Wait a few seconds, then check CloudWatch Logs
sleep 5
LOG_GROUP="/aws/lambda/s3-ingestion-trigger"
LOG_STREAM=$(aws logs describe-log-streams --log-group-name $LOG_GROUP \
  --order-by LastEventTime --descending --max-items 1 \
  --query 'logStreams[0].logStreamName' --output text)

aws logs get-log-events \
  --log-group-name $LOG_GROUP \
  --log-stream-name "$LOG_STREAM" \
  --query 'events[*].message' --output text
```

☒ Expected Output:

Log entry contains: `NEW OBJECT | bucket=my-datalake-lab-... | key=raw/source=csv/year=2024/month=01/sample_events.csv | size=... | time=...`

Step 8: Apply S3 Lifecycle policy for cost management

Configure a lifecycle rule to automatically transition objects to cheaper storage tiers over time and eventually expire them – essential for cost governance in a real data lake.

```
cat > /tmp/lifecycle.json << 'EOF'
{
  "Rules": [{
    "ID": "archive-and-expire-raw-data",
    "Status": "Enabled",
    "Filter": { "Prefix": "raw/" },
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" },
      { "Days": 180, "StorageClass": "DEEP_ARCHIVE" }
    ],
    "Expiration": { "Days": 365 },
```

```
"NoncurrentVersionExpiration": { "NoncurrentDays": 90 }
  ]]
}
EOF

aws s3api put-bucket-lifecycle-configuration \
  --bucket $BUCKET_NAME \
  --lifecycle-configuration file:///tmp/lifecycle.json

# Verify
aws s3api get-bucket-lifecycle-configuration --bucket $BUCKET_NAME
```

 **Note:** Lifecycle transitions take effect at midnight UTC. You won't see objects move immediately in a lab environment.

☒ **Expected Output:**

Lifecycle rule returned with ID 'archive-and-expire-raw-data' and all three Transitions configured.

 Lab 2: Kinesis Data Streams → Firehose → S3 Parquet Pipeline

Objective: Build a complete streaming ingestion pipeline: a Python producer sends clickstream events to Kinesis Data Streams, Kinesis Firehose reads from the stream, transforms records via Lambda (mask PII), converts JSON to Parquet using Glue schema, and lands files in S3.

Pre-requisites:

- Lab 1 completed – the S3 bucket (\$BUCKET_NAME) exists and BUCKET_NAME env var is set
- Python packages installed: `pip install boto3`
- Glue Data Catalog available in the same region (used for Parquet schema conversion)

Step 1: Create the Kinesis Data Stream

Create a 2-shard Kinesis Data Stream. Two shards give 2 MB/s ingest throughput and 4 MB/s read throughput – sufficient for this lab.

```
aws kinesis create-stream \  
  --stream-name clickstream-events \  
  --shard-count 2  
  
# Wait for stream to become ACTIVE (takes ~15 seconds)  
aws kinesis wait stream-exists --stream-name clickstream-events  
  
# Verify state  
aws kinesis describe-stream-summary \  
  --stream-name clickstream-events \  
  --query 'StreamDescriptionSummary.StreamStatus'
```

 Expected Output:

"ACTIVE" returned from describe-stream-summary.

Step 2: Create the Glue database and table for Parquet schema

Firehose uses a Glue Data Catalog table to understand the schema for JSON→Parquet conversion. Create the database and table definition matching the clickstream event structure.

```
# Create Glue database  
aws glue create-database \  
  --database-input '{"Name": "streaming_db", "Description": "Streaming data  
catalog"}'  
  
# Create the table with clickstream schema  
aws glue create-table \  
  --database-name streaming_db \  
  --table-input '{  
    "Name": "clickstream_events",  
    "StorageDescriptor": {  
      "Columns": [  
        {"Name": "user_id",      "Type": "string"},  
        {"Name": "event",        "Type": "string"},  
        {"Name": "page",         "Type": "string"},  
        {"Name": "session_id",   "Type": "string"},  
        {"Name": "ingestion_ts", "Type": "bigint"},  
      ]  
    }  
  }'
```

```

        {"Name": "email", "Type": "string"}
    ],
    "Location": "s3://'$BUCKET_NAME'/processed/clickstream/",
    "InputFormat": "org.apache.hadoop.mapred.TextInputFormat",
    "OutputFormat":
"org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat",
    "SerdeInfo": {
        "SerializationLibrary":
"org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe"
    }
},
    "PartitionKeys": [
        {"Name": "year", "Type": "string"},
        {"Name": "month", "Type": "string"}
    ],
    "TableType": "EXTERNAL_TABLE"
}'

```

☑ Expected Output:

Both commands return without error. Verify: `aws glue get-table --database-name streaming_db --name clickstream_events`

Step 3: Create the Lambda PII-masking transform function

Firehose invokes this Lambda synchronously for every batch of records. The function decodes base64, masks the email field, adds an ingestion timestamp, then re-encodes for Firehose to deliver.

```

cat > /tmp/firehose_transform.py << 'EOF'
import base64, json, time, logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    output = []
    for record in event['records']:
        try:
            # Decode payload
            payload = json.loads(base64.b64decode(record['data']))
            # Transform: add timestamp, mask PII
            payload['ingestion_ts'] = int(time.time())
            if 'email' in payload:
                local, domain = payload['email'].split('@') if '@' in
payload['email'] else (payload['email'], '')
                payload['email'] = local[0] + '***@' + domain if domain else
'***'
            result = 'Ok'
            data = base64.b64encode(json.dumps(payload).encode()).decode()
        except Exception as e:
            logger.error(f"Transform error: {e}")
            result = 'ProcessingFailed'
            data = record['data']
        output.append({'recordId': record['recordId'], 'result': result, 'data':
data})
    logger.info(f"Transformed {len(output)} records")

```

```

    return {'records': output}
EOF

cd /tmp && zip firehose_transform.zip firehose_transform.py

aws lambda create-function \
  --function-name firehose-pii-masker \
  --runtime python3.12 \
  --role $LAMBDA_ROLE_ARN \
  --handler firehose_transform.lambda_handler \
  --zip-file fileb:///tmp/firehose_transform.zip \
  --timeout 60

TRANSFORM_ARN=$(aws lambda get-function \
  --function-name firehose-pii-masker \
  --query 'Configuration.FunctionArn' --output text)
echo "Transform Lambda ARN: $TRANSFORM_ARN"

```

☑ Expected Output:

Lambda created successfully. FunctionArn printed.

Step 4: Create the Kinesis Firehose delivery stream

Create a Firehose delivery stream that reads from the KDS stream, transforms via Lambda, converts JSON to Parquet using the Glue schema, and writes to S3 with dynamic partitioning by year/month.

```

# First create an IAM role for Firehose
cat > /tmp/firehose-trust.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "firehose.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF

aws iam create-role \
  --role-name FirehoseDeliveryRole \
  --assume-role-policy-document file:///tmp/firehose-trust.json

# Attach required policies
aws iam attach-role-policy --role-name FirehoseDeliveryRole \
  --policy-arn arn:aws:iam::aws:policy/AmazonKinesisFullAccess
aws iam attach-role-policy --role-name FirehoseDeliveryRole \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
aws iam attach-role-policy --role-name FirehoseDeliveryRole \
  --policy-arn arn:aws:iam::aws:policy/AWSGlueConsoleFullAccess
aws iam attach-role-policy --role-name FirehoseDeliveryRole \
  --policy-arn arn:aws:iam::aws:policy/AWSLambda_FullAccess

```

```

FIREHOSE_ROLE=$(aws iam get-role --role-name FirehoseDeliveryRole \
  --query 'Role.Arn' --output text)

# Wait for role propagation
sleep 10

STREAM_ARN=$(aws kinesis describe-stream-summary \
  --stream-name clickstream-events \
  --query 'StreamDescriptionSummary.StreamARN' --output text)

aws firehose create-delivery-stream \
  --delivery-stream-name clickstream-delivery \
  --delivery-stream-type KinesisStreamAsSource \
  --kinesis-stream-source-configuration \
    "KinesisStreamARN=$STREAM_ARN,RoleARN=$FIREHOSE_ROLE" \
  --extended-s3-destination-configuration "{
    \"RoleARN\": \"\$FIREHOSE_ROLE\",
    \"BucketARN\": \"arn:aws:s3:::$BUCKET_NAME\",
    \"Prefix\":
    \"processed/clickstream/year=!{timestamp:yyyy}/month=!{timestamp:MM}/\",
    \"ErrorOutputPrefix\": \"errors/clickstream/\",
    \"BufferingHints\": {\"SizeInMBs\": 5, \"IntervalInSeconds\": 60},
    \"DataFormatConversionConfiguration\": {
      \"Enabled\": true,
      \"InputFormatConfiguration\": {\"Deserializer\": {\"OpenXJsonSerDe\":
    {}},
    \"OutputFormatConfiguration\": {\"Serializer\": {\"ParquetSerDe\": {}},
    \"SchemaConfiguration\": {
      \"RoleARN\": \"\$FIREHOSE_ROLE\",
      \"DatabaseName\": \"streaming_db\",
      \"TableName\": \"clickstream_events\",
      \"Region\": \"us-east-1\"
    }
  },
  \"ProcessingConfiguration\": {
    \"Enabled\": true,
    \"Processors\": [{\"Type\": \"Lambda\",
      \"Parameters\": [{\"ParameterName\": \"LambdaArn\", \"ParameterValue\":
    \"\$TRANSFORM_ARN\"}]
    }
  }
}

```

 **Note:** Firehose takes 1-2 minutes to become ACTIVE after creation. Wait before producing records.

☒ Expected Output:

DeliveryStreamARN returned. Check status: `aws firehose describe-delivery-stream --delivery-stream-name clickstream-delivery --query 'DeliveryStreamDescription.DeliveryStreamStatus'`

Step 5: Run the Python producer to send 100 clickstream events

The producer sends realistic clickstream records using `put_records()` (batch API, up to 500 records per call). Partition key = `user_id` ensures each user's events go to the same shard for ordered processing.

```
cat > /tmp/producer.py << 'EOF'
import boto3, json, random, time

kinesis = boto3.client('kinesis', region_name='us-east-1')
STREAM = 'clickstream-events'
PAGES = ['home', 'product', 'cart', 'checkout', 'confirmation', 'search']
EVENTS = ['view', 'click', 'scroll', 'purchase', 'add_to_cart', 'search']
USERS = [f"u{i:04d}" for i in range(1, 21)] # 20 unique users

records = []
for i in range(100):
    user = random.choice(USERS)
    records.append({
        'Data': json.dumps({
            'user_id': user,
            'event': random.choice(EVENTS),
            'page': random.choice(PAGES),
            'session_id': f"sess-{random.randint(1000, 9999)}",
            'email': f"{user}@example.com",
            'ts': int(time.time())
        }).encode(),
        'PartitionKey': user
    })

resp = kinesis.put_records(StreamName=STREAM, Records=records)
print(f"Sent: {len(records)} | Failed: {resp['FailedRecordCount']}")
EOF

python3 /tmp/producer.py
```

☑ Expected Output:

Sent: 100 | Failed: 0 — All records accepted by Kinesis. If FailedRecordCount > 0, re-run (transient throttle).

Step 6: Monitor Kinesis and Firehose metrics

Verify data is flowing through the pipeline by checking key CloudWatch metrics. The Firehose buffer window is 60 seconds, so allow up to 90 seconds before files appear in S3.

```
# Check Kinesis incoming records (last 5 minutes)
aws cloudwatch get-metric-statistics \
    --namespace AWS/Kinesis \
    --metric-name IncomingRecords \
    --dimensions Name=StreamName,Value=clickstream-events \
    --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
    --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
    --period 300 --statistics Sum

# Check Firehose incoming records
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/Firehose \
--metric-name IncomingRecords \
--dimensions Name=DeliveryStreamName,Value=clickstream-delivery \
--start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
--period 300 --statistics Sum
```

 **Note:** If Firehose metrics show 0, wait 2 minutes and retry. Firehose has up to 60s internal polling latency from KDS.

☒ **Expected Output:**

IncomingRecords Sum = 100 for both Kinesis and Firehose within 5 minutes.

Step 7: Validate Parquet files landed in S3

After the 60-second buffer window, Firehose flushes the batch to S3 as Parquet. List the output prefix and validate the file schema using Python.

```
# Wait for buffer to flush (60s minimum)
sleep 75

# List delivered Parquet files
aws s3 ls s3://$BUCKET_NAME/processed/clickstream/ --recursive

# Download and inspect one file
aws s3 cp $(aws s3 ls s3://$BUCKET_NAME/processed/clickstream/ --recursive \
| grep '.parquet' | head -1 | awk '{print "s3://" $BUCKET_NAME "/" $4}') \
/tmp/sample.parquet

# Read with Python
python3 -c "
import json
# Use pyarrow if available, otherwise show file size
try:
    import pyarrow.parquet as pq
    tbl = pq.read_table('/tmp/sample.parquet')
    print('Schema:', tbl.schema)
    print('Rows:', tbl.num_rows)
    print('First row:', tbl.to_pydict())
except ImportError:
    import os
    print('File size:', os.path.getsize('/tmp/sample.parquet'), 'bytes')
    print('Install pyarrow: pip install pyarrow')
"
```

☒ **Expected Output:**

*Parquet file listed under processed/clickstream/year=.../month=.../. Schema shows all 6 columns. Email field should show masked values (e.g., u***@example.com).*

 Lab 3: Glue Crawler & Data Catalog Discovery

Objective: Configure a Glue IAM role, create a Glue database, run a crawler against the S3 processed prefix, verify that tables and partitions are registered in the Data Catalog, then query the data using Athena to confirm partition pruning works.

Pre-requisites:

- Lab 1 completed – S3 bucket exists with data in the processed/ prefix
- Lab 2 completed (optional but recommended) – Parquet files exist in processed/clickstream/ prefix
- Athena must have an output location configured: s3://<bucket>/athena-results/

Step 1: Create the Glue service IAM role

Glue needs a role with trust to the Glue service principal, and permissions to read S3 and write to the Glue Data Catalog.

```
cat > /tmp/glue-trust.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "glue.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF

aws iam create-role \
  --role-name GlueLabRole \
  --assume-role-policy-document file:///tmp/glue-trust.json

# AWS managed policy for Glue service
aws iam attach-role-policy --role-name GlueLabRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole

# Inline policy granting S3 access to the specific bucket
cat > /tmp/glue-s3-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::$BUCKET_NAME",
      "arn:aws:s3:::$BUCKET_NAME/*"
    ]
  }]
}
EOF

aws iam put-role-policy \
  --role-name GlueLabRole \
  --policy-name S3DataLakeAccess \
  --policy-document file:///tmp/glue-s3-policy.json
```

```
GLUE_ROLE=$(aws iam get-role --role-name GlueLabRole --query 'Role.Arn' --output text)
echo "Glue Role ARN: $GLUE_ROLE"
```

☑ Expected Output:

Role ARN printed. Verify: `aws iam get-role --role-name GlueLabRole --query 'Role.RoleName'`

Step 2: Create the Glue Data Catalog database

A Glue database is a logical container for tables. Tables within a database typically share a common data domain or source system.

```
aws glue create-database \
  --database-input '{
    "Name": "lab_datalake_db",
    "Description": "Day 1 lab data lake - S3 ingestion and streaming data"
  }'

# Verify
aws glue get-database --name lab_datalake_db \
  --query 'Database.{Name:Name, Description:Description}'
```

☑ Expected Output:

{"Name": "lab_datalake_db", "Description": "Day 1 lab data lake - S3 ingestion and streaming data"}

Step 3: Create and configure the Glue Crawler

Configure a crawler targeting the entire processed/ prefix. The CRAWL_NEW_FOLDERS_ONLY recrawl policy means subsequent runs only scan new partitions, not the full dataset – significantly reducing cost and runtime.

```
aws glue create-crawler \
  --name "lab-processed-crawler" \
  --role "$GLUE_ROLE" \
  --database-name "lab_datalake_db" \
  --targets '{
    \"S3Targets\": [{
      \"Path\": \"s3://$BUCKET_NAME/processed/\",
      \"Exclusions\": [\"**/_temporary/**\", \"**/errors/**\", \"**/.keep\"]
    }]
  }' \
  --schema-change-policy '{
    \"UpdateBehavior\": \"UPDATE_IN_DATABASE\",
    \"DeleteBehavior\": \"LOG\"
  }' \
  --recrawl-policy '{\"RecrawlBehavior\": \"CRAWL_NEW_FOLDERS_ONLY\"}' \
  --configuration '{
    \"Version\": 1.0,
    \"CrawlerOutput\": {
      \"Partitions\": {\"AddOrUpdateBehavior\": \"InheritFromTable\"}
    }
  }'
```

```
}'  
  
# Verify creation  
aws glue get-crawler --name "lab-processed-crawler" \  
  --query 'Crawler.{Name:Name, State:State, Role:Role}'
```

☒ **Expected Output:**

```
{"Name": "lab-processed-crawler", "State": "READY", "Role": "GlueLabRole"}
```

Step 4: Run the crawler and monitor progress

Start the crawler and poll until it returns to READY state. Typical run time for a small S3 prefix is 1–3 minutes.

```
# Start the crawler  
aws glue start-crawler --name "lab-processed-crawler"  
echo "Crawler started at: $(date)"  
  
# Poll until READY (check every 15 seconds)  
while true; do  
  STATE=$(aws glue get-crawler --name "lab-processed-crawler" \  
    --query 'Crawler.State' --output text)  
  echo "$(date): State = $STATE"  
  if [ "$STATE" = "READY" ]; then  
    echo "Crawler finished!"  
    break  
  fi  
  sleep 15  
done  
  
# Check last crawl metrics  
aws glue get-crawler --name "lab-processed-crawler" \  
  --query 'Crawler.LastCrawl'
```

☒ **Expected Output:**

Crawler State transitions: RUNNING → STOPPING → READY. LastCrawl shows Status: SUCCEEDED and TablesCreated > 0.

Step 5: Inspect the discovered tables and partitions

Verify the Data Catalog tables were created correctly. Check the column schema, storage format, and discovered partitions.

```
# List tables in the database  
aws glue get-tables --database-name lab_datalake_db \  
  --query 'TableList[*].{Name:Name, Location:StorageDescriptor.Location, Format:StorageDescriptor.SerdeInfo.SerializationLibrary}'  
  
# Get column details for the clickstream table  
aws glue get-table --database-name lab_datalake_db --name clickstream_events \  
  --query 'Table.StorageDescriptor.Columns[*].{Name:Name, Type:Type}'
```

```
# List discovered partitions
aws glue get-partitions --database-name lab_datalake_db --name
clickstream_events \
  --query 'Partitions[*].Values'
```

☒ **Expected Output:**

Table listed with Parquet SerDe. Columns show user_id, event, page, etc. Partitions listed show year/month values.

Step 6: Configure Athena output location

Athena requires an S3 location for query results. Create the prefix and set it as the Athena workgroup output location.

```
# Create the Athena results prefix
aws s3api put-object --bucket $BUCKET_NAME --key athena-results/.keep

# Set the Athena primary workgroup output location
aws athena update-work-group \
  --work-group primary \
  --configuration-updates \
    "ResultConfigurationUpdates={OutputLocation=s3://$BUCKET_NAME/athena-
results/}"
```

☒ **Expected Output:**

No error output means the workgroup was updated successfully.

Step 7: Query the catalog table in Athena and verify partition pruning

Run queries against the discovered table and verify that Athena uses partition metadata to scan only the relevant data (partition pruning). Compare data scanned with and without a partition filter.

```
# Helper: run Athena query and print results + data scanned
run_query() {
  QID=$(aws athena start-query-execution \
    --query-string "$1" \
    --query-execution-context "Database=lab_datalake_db" \
    --result-configuration "OutputLocation=s3://$BUCKET_NAME/athena-results/" \
    --query 'QueryExecutionId' --output text)
  sleep 5
  aws athena get-query-execution --query-execution-id $QID \
    --query 'QueryExecution.{Status:Status.State,
DataScanned:Statistics.DataScannedInBytes}'
  aws athena get-query-results --query-execution-id $QID \
    --query 'ResultSet.Rows[0:5]'
}

# Query 1: Count all records (no partition filter)
echo "=== Query 1: Full table scan ==="
run_query "SELECT COUNT(*) as total_events FROM clickstream_events"

# Query 2: Filter by year partition (partition pruning active)
```

```
echo "=== Query 2: Partition-pruned query ==="
run_query "SELECT year, month, COUNT(*) as events FROM clickstream_events WHERE
year='2024' AND month='01' GROUP BY year, month"
```

 **Note:** If the table has no data yet, run: `MSCK REPAIR TABLE clickstream_events;` in Athena console to refresh all partitions.

 Expected Output:

Query 1 scans more bytes. Query 2 scans significantly fewer bytes (only the matching partition). Both return row counts matching the data you uploaded.

Step 8: Add a new partition manually without re-running the crawler

For production pipelines, adding partitions via the Glue API is far faster than re-running a full crawler. This is the preferred pattern for incremental partition registration.

```
# Upload data for a new day partition
cat > /tmp/new_day.csv << 'EOF'
user_id,event,page,session_id,ingestion_ts,email
u0001,view,home,sess-1111,1705276800,u***@example.com
u0002,click,product,sess-2222,1705276860,u***@example.com
EOF
aws s3 cp /tmp/new_day.csv \
    "s3://$BUCKET_NAME/processed/clickstream/year=2024/month=01/day=16/data.csv"

# Add partition to catalog WITHOUT re-running crawler
python3 << 'EOF'
import boto3
glue = boto3.client('glue', region_name='us-east-1')
import os

bucket = os.environ.get('BUCKET_NAME', 'YOUR_BUCKET_NAME')
glue.batch_create_partition(
    DatabaseName='lab_datalake_db',
    TableName='clickstream_events',
    PartitionInputList=[{
        'Values': ['2024', '01'],
        'StorageDescriptor': {
            'Location':
f's3://{bucket}/processed/clickstream/year=2024/month=01/day=16/',
            'InputFormat': 'org.apache.hadoop.mapred.TextInputFormat',
            'OutputFormat':
'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat',
            'SerdeInfo': {'SerializationLibrary':
'org.openx.data.jsonserde.JsonSerDe'}
        }
    }]
)
print("Partition added successfully")
EOF
```

 Expected Output:

Partition added successfully. Query Athena immediately – the new partition is visible without any crawler run.

 Lab 4: EventBridge + Step Functions Orchestrated Ingestion Pipeline

Objective: Build a production-grade orchestration layer: an EventBridge rule triggers a Step Functions state machine on a schedule. The state machine validates input, triggers a Glue crawler, waits for completion, and sends SNS notifications on success or failure. Test error-handling paths.

Pre-requisites:

- Labs 1–3 completed – S3 bucket, Lambda role, Glue crawler, and SNS topic available
- SNS: create a topic before starting this lab: `aws sns create-topic --name ingestion-alerts`
- Subscribe your email to the SNS topic for notification testing (SNS console > Subscriptions)

Step 1: Create the SNS topic and subscribe your email

The state machine sends pipeline success/failure alerts via SNS. Create the topic and subscribe your email so you can verify notifications end-to-end.

```
# Create SNS topic
SNS_ARN=$(aws sns create-topic --name ingestion-alerts \
  --query 'TopicArn' --output text)
echo "SNS ARN: $SNS_ARN"

# Subscribe your email (replace with your actual email)
aws sns subscribe \
  --topic-arn $SNS_ARN \
  --protocol email \
  --notification-endpoint "you@example.com"

echo "Check your email and confirm the subscription before continuing."
echo "SNS_ARN=$SNS_ARN"
```

 **Note:** You must click the confirmation link in the email before SNS will deliver messages to your address.

 Expected Output:

Subscription ARN returned with status PendingConfirmation. Confirm via email before proceeding.

Step 2: Create the IAM role for Step Functions

Step Functions needs permissions to invoke Lambda, start Glue crawlers, query Glue state, publish to SNS, and start its own executions (for nested state machines).

```
cat > /tmp/sfn-trust.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "states.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF
```

```
aws iam create-role \
  --role-name StepFunctionsIngestionRole \
  --assume-role-policy-document file:///tmp/sfn-trust.json

# Inline policy with all required permissions
cat > /tmp/sfn-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["lambda:InvokeFunction"],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": ["glue:StartCrawler", "glue:GetCrawler", "glue:StopCrawler"],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": ["sns:Publish"],
      "Resource": "$SNS_ARN"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:HeadObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::$BUCKET_NAME", "arn:aws:s3:::$BUCKET_NAME/*"]
    },
    {
      "Effect": "Allow",
      "Action":
["logs:CreateLogGroup", "logs:CreateLogDelivery", "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}
EOF

aws iam put-role-policy \
  --role-name StepFunctionsIngestionRole \
  --policy-name SFNIngestionPolicy \
  --policy-document file:///tmp/sfn-policy.json

SFN_ROLE=$(aws iam get-role --role-name StepFunctionsIngestionRole \
  --query 'Role.Arn' --output text)
echo "Step Functions Role ARN: $SFN_ROLE"
```

☑ Expected Output:

Role ARN printed. Both CreateRole and PutRolePolicy succeed without error.

Step 3: Create the input validation Lambda function

This function checks whether the S3 object specified in the pipeline input actually exists. It returns {valid: true/false} which the state machine uses for branching.

```
cat > /tmp/validate_input.py << 'EOF'
import boto3, json, logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

s3 = boto3.client('s3')

def lambda_handler(event, context):
    bucket = event.get('bucket')
    key = event.get('key')
    if not bucket or not key:
        raise ValueError("Missing 'bucket' or 'key' in input")
    try:
        s3.head_object(Bucket=bucket, Key=key)
        logger.info(f"Validated: s3://{bucket}/{key}")
        return {'event': event, 'valid': True, 'message': f"Object exists: {key}"}
    except s3.exceptions.ClientError as e:
        if e.response['Error']['Code'] == '404':
            logger.warning(f"Object not found: s3://{bucket}/{key}")
            raise Exception(f"ValidationError: Object not found: s3://{bucket}/{key}")
        raise
EOF

cd /tmp && zip validate_input.zip validate_input.py

aws lambda create-function \
  --function-name pipeline-validate-input \
  --runtime python3.12 \
  --role $LAMBDA_ROLE_ARN \
  --handler validate_input.lambda_handler \
  --zip-file fileb:///tmp/validate_input.zip \
  --timeout 30

VALIDATE_ARN=$(aws lambda get-function --function-name pipeline-validate-input \
  --query 'Configuration.FunctionArn' --output text)
echo "Validate Lambda ARN: $VALIDATE_ARN"
```

☑ Expected Output:

Lambda created with State=Active. FunctionArn printed.

Step 4: Define and deploy the Step Functions state machine (ASL)

The state machine implements the full pipeline: Validate → StartCrawler → Poll crawler state → (Success: NotifySuccess) / (Failure: NotifyFailure). It includes retry logic on transient AWS service errors.

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

cat > /tmp/pipeline_asl.json << EOF
{
```

```

    "Comment": "Ingestion Pipeline: validate input, run Glue crawler, notify
result",
    "StartAt": "ValidateInput",
    "States": {
        "ValidateInput": {
            "Type": "Task",
            "Resource": "arn:aws:states:::lambda:invoke",
            "Parameters": {
                "FunctionName": "$VALIDATE_ARN",
                "Payload.$": "$"
            },
            "ResultSelector": { "body.$": "States.StringToJson(States.Format('{}',
States.JsonToString($.Payload)))" },
            "ResultPath": "$.validation",
            "Retry": [{
                "ErrorEquals": ["Lambda.ServiceException", "Lambda.AWSLambdaException"],
                "IntervalSeconds": 5, "MaxAttempts": 3, "BackoffRate": 2
            }],
            "Catch": [{
                "ErrorEquals": ["States.ALL"],
                "ResultPath": "$.error",
                "Next": "NotifyFailure"
            }],
            "Next": "StartGlueCrawler"
        },
        "StartGlueCrawler": {
            "Type": "Task",
            "Resource": "arn:aws:states:::aws-sdk:glue:startCrawler",
            "Parameters": { "Name": "lab-processed-crawler" },
            "ResultPath": null,
            "Catch": [{
                "ErrorEquals": ["Glue.CrawlerRunningException"],
                "ResultPath": "$.error",
                "Next": "WaitForCrawler"
            }],
            "ErrorEquals": ["States.ALL"],
            "ResultPath": "$.error",
            "Next": "NotifyFailure"
        },
        "WaitForCrawler": {
            "Type": "Wait",
            "Seconds": 20,
            "Next": "CheckCrawlerStatus"
        },
        "CheckCrawlerStatus": {
            "Type": "Task",
            "Resource": "arn:aws:states:::aws-sdk:glue:getCrawler",
            "Parameters": { "Name": "lab-processed-crawler" },
            "ResultPath": "$.crawlerResult",
            "Next": "IsCrawlerDone"
        },
        "IsCrawlerDone": {
            "Type": "Choice",
            "Choices": [

```

```

        { "Variable": "$.crawlerResult.Crawler.State", "StringEquals": "READY",
"Next": "CheckLastCrawlStatus" },
        { "Variable": "$.crawlerResult.Crawler.State", "StringEquals":
"STOPPING", "Next": "WaitForCrawler" }
    ],
    "Default": "WaitForCrawler"
},
"CheckLastCrawlStatus": {
    "Type": "Choice",
    "Choices": [{
        "Variable": "$.crawlerResult.Crawler.LastCrawl.Status",
        "StringEquals": "SUCCEEDED",
        "Next": "NotifySuccess"
    }],
    "Default": "NotifyFailure"
},
"NotifySuccess": {
    "Type": "Task",
    "Resource": "arn:aws:states:::sns:publish",
    "Parameters": {
        "TopicArn": "$SNS_ARN",
        "Subject": "☑ Ingestion Pipeline Succeeded",
        "Message.$": "States.Format('Pipeline completed for bucket={} key={}.
Crawler finished successfully.', $.bucket, $.key)"
    },
    "End": true
},
"NotifyFailure": {
    "Type": "Task",
    "Resource": "arn:aws:states:::sns:publish",
    "Parameters": {
        "TopicArn": "$SNS_ARN",
        "Subject": "✗ Ingestion Pipeline FAILED",
        "Message.$": "States.Format('Pipeline FAILED for bucket={} key={}.
Error: {}', $.bucket, $.key, States.JsonToString($.error))"
    },
    "End": true
}
}
}
EOF

# Deploy the state machine
SFN_ARN=$(aws stepfunctions create-state-machine \
--name IngestionPipeline \
--definition file:///tmp/pipeline_asl.json \
--role-arn $SFN_ROLE \
--query 'stateMachineArn' --output text)

echo "State Machine ARN: $SFN_ARN"

```

☑ Expected Output:

stateMachineArn returned in the format arn:aws:states:us-east-1:123456789012:stateMachine:IngestionPipeline

Step 5: Test the happy path (valid S3 object)

Start a manual execution with a known-good S3 object. Monitor execution progress and verify the success notification is sent.

```
# Start an execution with a valid S3 key
EXEC_ARN=$(aws stepfunctions start-execution \
  --state-machine-arn $SFN_ARN \
  --name "test-happy-path-$(date +%s)" \
  --input "{
    \"bucket\": \"$BUCKET_NAME\",
    \"key\": \"raw/source=csv/year=2024/month=01/sample_events.csv\"
  }" \
  --query 'executionArn' --output text)

echo "Execution ARN: $EXEC_ARN"

# Poll for completion
while true; do
  STATUS=$(aws stepfunctions describe-execution \
    --execution-arn $EXEC_ARN \
    --query 'status' --output text)
  echo "$(date): Status = $STATUS"
  [ "$STATUS" = "RUNNING" ] || break
  sleep 10
done

# Get execution history summary
aws stepfunctions get-execution-history \
  --execution-arn $EXEC_ARN \
  --query 'events[*].{Type:type, Timestamp:timestamp}' \
  --output table
```

 **Note:** The entire execution takes 60-120 seconds because WaitForCrawler polls every 20 seconds. You can monitor visually in the Step Functions console under 'Graph view'.

☑ Expected Output:

Status transitions from RUNNING to SUCCEEDED. Execution history shows all states visited in order. Check your email for the success SNS notification.

Step 6: Test the failure path (non-existent S3 object)

Start an execution with an invalid S3 key. The ValidateInput Lambda throws an exception, the Catch block routes to NotifyFailure, and a failure SNS notification is sent.

```
EXEC_ARN_FAIL=$(aws stepfunctions start-execution \
  --state-machine-arn $SFN_ARN \
  --name "test-failure-path-$(date +%s)" \
  --input "{
    \"bucket\": \"$BUCKET_NAME\",
    \"key\": \"raw/non-existent-file-that-does-not-exist.csv\"
  }" \
  --query 'executionArn' --output text)
```

```

echo "Failure test Execution ARN: $EXEC_ARN_FAIL"

# Poll for completion (should be fast - Lambda fails immediately)
while true; do
    STATUS=$(aws stepfunctions describe-execution \
        --execution-arn $EXEC_ARN_FAIL \
        --query 'status' --output text)
    echo "$(date): Status = $STATUS"
    [ "$STATUS" = "RUNNING" ] || break
    sleep 5
done

# Show the error that was caught
aws stepfunctions describe-execution \
    --execution-arn $EXEC_ARN_FAIL \
    --query 'output'

```

☒ **Expected Output:**

Execution reaches SUCCEEDED status (because NotifyFailure is an End state, not a Fail state). Output JSON contains the error details. Check email for failure notification.

Step 7: Create the EventBridge scheduled rule for production

Schedule the pipeline to run automatically every night at 2:00 AM UTC. The EventBridge rule invokes Step Functions with a fixed input payload.

```

# Create IAM role for EventBridge to invoke Step Functions
cat > /tmp/eb-trust.json << 'EOF'
{
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Allow",
        "Principal": { "Service": "events.amazonaws.com" },
        "Action": "sts:AssumeRole"
    }]
}
EOF

aws iam create-role --role-name EventBridgeSFNRole \
    --assume-role-policy-document file:///tmp/eb-trust.json

aws iam put-role-policy --role-name EventBridgeSFNRole \
    --policy-name InvokeSFN \
    --policy-document "{
    \"Version\": \"2012-10-17\",
    \"Statement\": [{
        \"Effect\": \"Allow\",
        \"Action\": \"states:StartExecution\",
        \"Resource\": \"\${SFN_ARN}\"
    }]
}"

```

```
EB_ROLE=$(aws iam get-role --role-name EventBridgeSFNRole --query 'Role.Arn' --
output text)

# Create the daily schedule rule (2 AM UTC)
aws events put-rule \
  --name "daily-ingestion-pipeline" \
  --schedule-expression "cron(0 2 * * ? *)" \
  --state ENABLED \
  --description "Trigger nightly data ingestion pipeline at 2AM UTC"

# Add Step Functions as the target
aws events put-targets \
  --rule "daily-ingestion-pipeline" \
  --targets "[{
    \"Id\": \"1\",
    \"Arn\": \"\${SFN_ARN}\",
    \"RoleArn\": \"\${EB_ROLE}\",
    \"Input\": \"{\\\"bucket\\\": \\\"$\${BUCKET_NAME}\\\", \\\"key\\\":
\\\"raw/source=csv/year=2024/month=01/sample_events.csv\\\"}\"
  }]"

echo "Scheduled rule created: daily-ingestion-pipeline"
echo "Next run: tomorrow at 02:00 UTC"
```

☑ Expected Output:

Rule created and target added. Verify: `aws events list-targets-by-rule --rule daily-ingestion-pipeline`

Step 8: Verify end-to-end pipeline via CloudWatch dashboard

Create a simple CloudWatch dashboard to monitor the pipeline's key health metrics in one place.

```
aws cloudwatch put-dashboard \
  --dashboard-name "IngestionPipelineDashboard" \
  --dashboard-body "{
    \"widgets\": [
      {
        \"type\": \"metric\",
        \"properties\": {
          \"title\": \"Step Functions Executions\",
          \"metrics\": [

[\"AWS/States\", \"ExecutionsStarted\", \"StateMachineArn\", \"\${SFN_ARN}\"],

[\"AWS/States\", \"ExecutionsSucceeded\", \"StateMachineArn\", \"\${SFN_ARN}\"],

[\"AWS/States\", \"ExecutionsFailed\", \"StateMachineArn\", \"\${SFN_ARN}\"]
          ],
          \"period\": 300, \"stat\": \"Sum\", \"view\": \"timeSeries\"
        }
      },
      {
        \"type\": \"metric\",
        \"properties\": {
```

```
\ "title\": \ "Kinesis Incoming Records\ ",
\ "metrics\ ": [[
  \ "AWS/Kinesis\ ", \ "IncomingRecords\ ",
  \ "StreamName\ ", \ "clickstream-events\ "
]],
\ "period\ ": 60, \ "stat\ ": \ "Sum\ ", \ "view\ ": \ "timeSeries\ "
}
]
}"
```

```
echo "Dashboard created. View at:"
echo "https://console.aws.amazon.com/cloudwatch/home?region=us-east-1#dashboards:name=IngestionPipelineDashboard"
```

☒ **Expected Output:**

Dashboard created. Open the URL in the AWS Console to see your pipeline metrics in real-time.

7. Exam Tips & Common Scenarios

7.1 Key Decision Patterns

When the Exam Says... Use This Service
'Real-time streaming' or 'millisecond latency' → Kinesis Data Streams
'Load streaming data to S3/Redshift without code' → Kinesis Data Firehose
'Scheduled batch ingestion from on-premises' → AWS DataSync or S3 Transfer Family
'Automatically discover schema from S3' → Glue Crawler
'Orchestrate multi-step pipeline with retries' → Step Functions
'Trigger pipeline on S3 file arrival' → S3 Event Notifications + Lambda/EventBridge
'Query S3 data without loading to warehouse' → Athena + Glue Catalog
'Need SFTP endpoint for external partners' → AWS Transfer Family

7.2 Common Trap Questions

Scenario	Wrong Answer	Correct Answer
Need to ingest 50GB CSV nightly from on-prem	Kinesis Data Streams	AWS DataSync → S3
Multiple consumers need same stream data	Multiple Firehose streams	KDS with Enhanced Fan-Out
Convert streaming JSON to Parquet in S3	Lambda after Firehose	Firehose built-in format conversion
New S3 partitions not visible in Athena	Re-run full Glue Crawler	MSCK REPAIR TABLE or batch add partition
Orchestrate Glue job + crawler sequence	Lambda polling loop	Glue Workflow or Step Functions
Ingest from Salesforce to S3 daily	Custom Kinesis producer	AppFlow or Firehose partner source

7.3 Architecture Anti-Patterns to Avoid

- Don't use Kinesis Streams for batch bulk transfers (high cost, low efficiency)
- Don't skip partition strategy on S3 -- full table scans in Athena are expensive
- Don't use Firehose when you need sub-second latency -- buffer minimum is 60 seconds
- Don't rely on S3 eventual consistency for immediate post-write reads (strong consistency since Dec 2020)
- Don't create one shard per record type -- use high-cardinality partition keys to distribute load

- Don't use Glue Crawlers as a trigger mechanism -- use S3 Events or EventBridge instead

7.4 Day 1 Quick Reference Summary

Topic	Service	Key Exam Fact
Batch ingestion landing	S3	Hive partition format auto-discovered by Glue
File transfer (on-prem)	DataSync / Transfer Family	DataSync: NFS/SMB; Transfer: SFTP
Real-time streaming	Kinesis Data Streams	1MB/s in, 2MB/s out per shard
Stream to S3 (no code)	Kinesis Firehose	Min buffer: 60s or 1MB
Schema discovery	Glue Crawler	Writes to Glue Data Catalog
Query catalog tables	Athena	Serverless; charged per TB scanned
Event-driven triggers	S3 Events + EventBridge	ObjectCreated → Lambda/SQS/SNS
Complex orchestration	Step Functions	ASL, retries, parallel, catch
Glue-only orchestration	Glue Workflows	Conditional triggers between jobs

8. Further Reading & Resources

- AWS Documentation: Amazon Kinesis Data Streams Developer Guide -- covers shard management, enhanced fan-out, and KCL
- AWS Documentation: AWS Glue Developer Guide -- Crawler configuration, Data Catalog, schema evolution
- AWS Documentation: AWS Step Functions Developer Guide -- Amazon States Language reference, SDK integrations
- AWS Blog: Best Practices for Building a Data Lake on S3 -- partitioning, lifecycle policies, access patterns
- AWS re:Invent Session: Building a Serverless Data Lake (DAT325) -- end-to-end architecture walkthrough
- AWS Exam Guide: DEA-C01 -- Domain 1 accounts for 34% of exam questions; focus on service selection and integration patterns